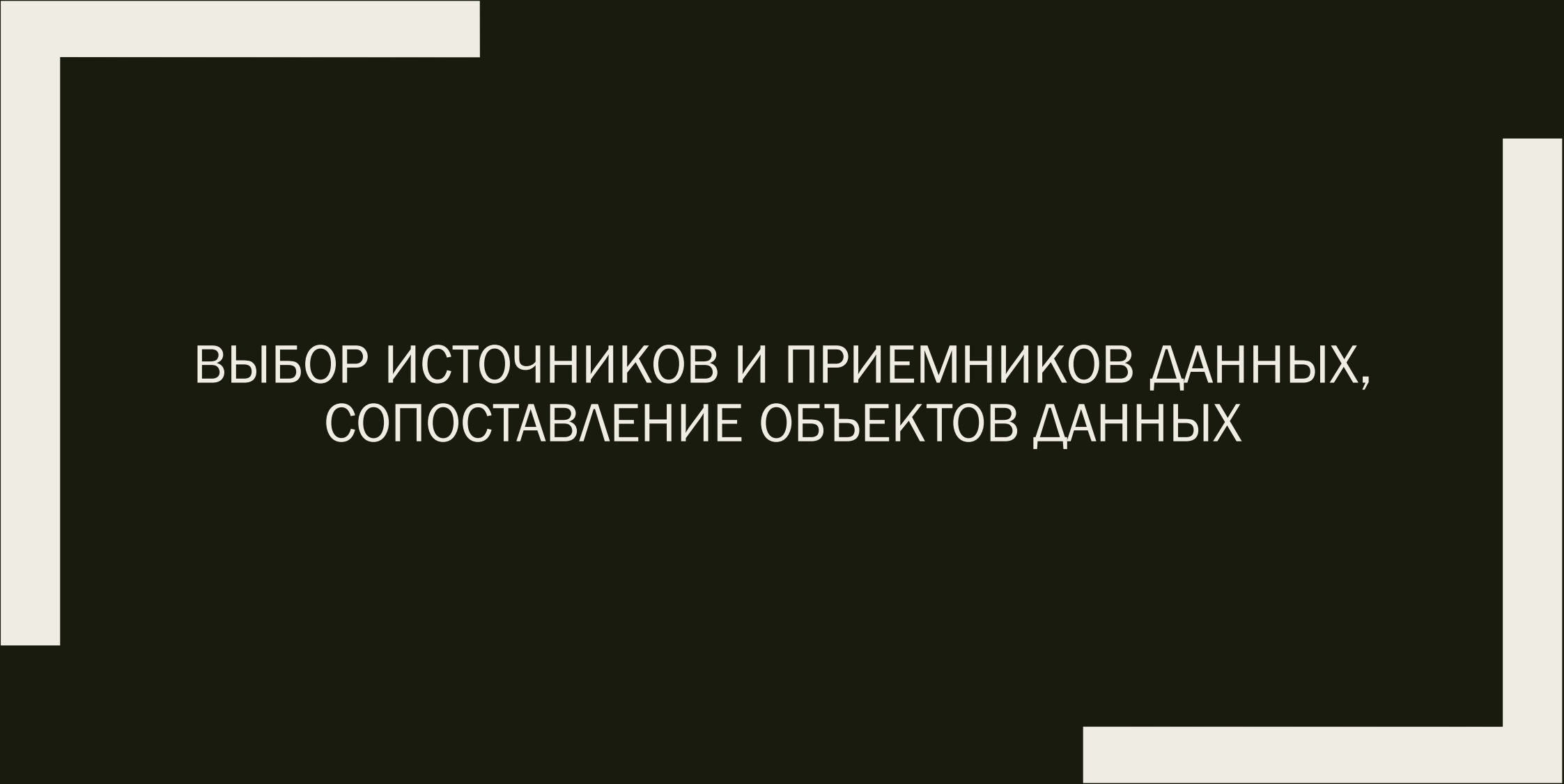




ВЫБОР ИСТОЧНИКОВ И ПРИЕМНИКОВ ДАННЫХ,
СОПОСТАВЛЕНИЕ ОБЪЕКТОВ ДАННЫХ

Интеграция приложений

Интеграция приложений — это стратегический подход к объединению информационных систем, который обеспечивает возможность обмена информацией и поддержания распределенных бизнес-процессов. Интеграция информационных систем дает предприятию такие несомненные конкурентные преимущества, как:

- ведение бизнеса в режиме реального времени с использованием событийно управляемых сценариев;
- владение достоверной, полной и своевременно полученной информацией.

Основная задача интеграции

Задача интеграции — обеспечить эффективный, надежный и безопасный обмен данными между различными программными продуктами, изначально не предназначенными для совместной работы.

Основные движущие силы интеграции

Основными движущими силами интеграции являются:

- **электронный бизнес** — интеграция унаследованных информационных систем, поддерживающих ключевую функциональность, с Web-приложениями (Web-сервисами и порталами) с целью получения доступа к бизнес-функциям через Интернет;
- **управление цепями поставок** — интеграция разрозненных систем управления заказами, MRP-систем, систем календарного планирования, систем транспортного менеджмента с целью прямого обмена информацией между покупателями и поставщиками в режиме реального времени;

Основные движущие силы интеграции

- управление взаимоотношениями с клиентами — получение единого консолидированного представления о клиенте путем объединения данных о нем, распределенных между несколькими изолированными приложениями (интеграция клиентских баз данных, call-центров, интернет-сервисов);
- внедрение ERP — интеграция модулей ERP-систем, поддерживающих базовую функциональность, со специализированным программным обеспечением, используемым организацией;
- электронное правительство — интеграция унаследованных backend систем с front-end Web-приложениями, организация обмена данными между правительственными учреждениями;

Основные движущие силы интеграции

- самообслуживание клиентов — возможность клиентов самостоятельно выполнять действия, традиционно являющиеся функцией обслуживающего персонала, требует интеграции пользовательских приложений с back-end-системами;
- Business Intelligence — сбор данных из различных приложений и источников в хранилище данных с целью их обработки и анализа;

Основные движущие силы интеграции

- управление знаниями — обеспечение доступа в режиме реального времени к корпоративному контенту, распределенному между многочисленными источниками, с целью управления знаниями в масштабах предприятия;
- облачные технологии — интеграция существующих бизнес-приложений с облачными приложениями и сервисами;
- аутсорсинг бизнес-процессов — интеграция с информационными системами партнеров.

Основные бизнес-выгоды

Основные бизнес-выгоды, которые предприятие может получить в случае успешной реализации интеграционного проекта:

- улучшение качества поддержки и обслуживания клиентов;
- автоматизация бизнес-процессов;
- уменьшение производственного цикла;
- сокращение количества ошибок обработки данных;
- прозрачность процессов;
- уменьшение стоимости транзакций;
- оптимизация логистических процессов;
- более тесное взаимодействие с бизнес-партнерами;
- быстрое внедрение новых бизнес-сервисов;
- сохранение инвестиций в информационные технологии

Технические проблемы при интеграции

Можно выделить несколько групп проблем, с которыми сталкиваются разработчики интеграционных решений:

1) технические проблемы:

- риск задержки и потери информации, вызванный ненадежностью сетей передачи данных (обеспечивая связь между отдельными частями интеграционного решения, необходимо передавать информацию между устройствами по телефонным линиям, локальным сетям, через маршрутизаторы, общедоступные сети и каналы спутниковой связи);
- недостаточная скорость передачи данных (объективно время доставки данных через компьютерную сеть всегда больше времени вызова локального метода, поэтому интеграционное решение всегда работает медленнее локального приложения);
- необходимость учитывать все различия между приложениями (разные языки программирования, на которых написаны приложения, разные платформы, разный формат данных);
- ограниченный контроль над интегрируемыми приложениями, представляющими собой, например, унаследованные коммерческие приложения с закрытой структурой баз данных или плохо документированные системы, разработанные ИТ-департаментом;
- необходимость поддерживать интеграционное решение, то есть адаптировать его к изменениям в интегрируемых приложениях.

Технические проблемы при интеграции

Зачастую изменения в одной системе влекут за собой непредсказуемые изменения в других;

2) методологические проблемы:

- отсутствие корректного формата или семантического слоя для слияния двух и более несопоставимых наборов данных. Устранение семантических различий между приложениями — одна из наиболее сложных и неформализуемых задач интеграции. Действительно, одна и та же сущность (например, «Клиент») может иметь несколько различных семантик, ограничений и допущений в каждой системе. Эта проблема в англоязычной литературе получила название семантического диссонанса;
- необходимость наличия методологии для документирования таких технических аспектов интеграции, как определение записей, структур данных, интерфейсов и потоков данных в масштабах всей организации;
- необходимость определения правил и методов согласования данных во всех интеграционных проектах (правил устранения семантического диссонанса);

Технические проблемы при интеграции

3) организационные проблемы:

- недостаток доверия к корректности информации;
- интеграция приложений в большинстве случаев влечет за собой пересмотр корпоративной политики компании. Объединение информационных систем (например, CRM-системы и системы бухгалтерского учета) требует изменений в порядке взаимодействия между использующими эти системы отделами (отделом маркетинга и бухгалтерией);
- при интеграции ключевых бизнес-функций компании ее деятельность становится зависимой от функционирования интеграционного решения, сбой в работе которого может принести компании существенные убытки (ошибочные платежи, потерянные заказы и т.д.).

Подход, основанный на использовании шаблонов

- Связывание разнородных корпоративных приложений требует особой инфраструктуры для перемещения данных между системами.
- В некоторых случаях необходимо не только перемещать данные, но и поддерживать определенную бизнес-логику взаимодействия приложений, то есть автоматизировать распределенный бизнес-процесс или обеспечить единую точку доступа к информации, предоставляемой отдельными приложениями.
- Не существует универсального интеграционного решения, которое могло бы подойти всем. Каждое интеграционное решение уникально и требует применения различных подходов к объединению приложений. Однако, несмотря на многообразие задач интеграции, и способов связывания приложений, различными авторами предпринимались
- попытки выделить ряд базовых классов интеграционных решений, используя язык шаблонов.

Шаблон

Шаблон выполняет роль референтной модели и представляет собой абстрактное описание типовой задачи и способов ее решения.

Развитию подхода, основанного на концепции шаблонов, способствовали работы посвященные проектированию архитектуры корпоративных информационных систем. **Шаблон** содержит описание типового решения, которое должно быть принято на этапе проектирования, и его обоснование. Общепризнано, что использование шаблонов — один из наиболее эффективных способов документирования экспертных знаний.

Шаблон

Для того чтобы тот или иной шаблон можно было использовать для решения практических задач, он должен содержать: обоснование постановки задачи, описание возможных способов ее решения и объяснение преимуществ предлагаемого подхода. Язык шаблонов — это набор связанных между собой шаблонов. Шаблоны интеграции помогают систематизировать знания в области интеграции информационных систем.

Каждый эксперт или экспертное сообщество использует свой язык шаблонов и стиль изложения.

Как правило, описание шаблона строится из следующих элементов:

- имя — имя шаблона, отражающее его назначение;
- пиктограмма — визуальное представление шаблона;
- контекст — описание ситуации, в которой возможно применение данного шаблона;
- возможные проблемы — описание типовых проблем, возникающих при реализации данного стиля интеграции;
- пример реализации шаблона, описание варианта технического решения.

Шаблон

Следует понимать, что система шаблонов образует иерархию, на верхнем уровне которой находятся шаблоны, определяющие архитектуру промежуточного слоя. Для любой архитектуры слоя `middleware` могут быть использованы различные способы связывания приложений, равно как и для каждого способа связывания могут быть предложены различные топологии интеграционного решения.

Архитектура промежуточного слоя

С точки зрения архитектуры интеграционного решения можно выделить три базовых шаблона промежуточного слоя:

- агрегация сущностей (Entity Aggregation);
- интеграция процессов (Process Integration);
- интеграция, ориентированная на портал (Portal Integration);

Основные характеристики базовых шаблонов

Шаблон промежуточного слоя	Решаемая проблема
Агрегация сущностей	Как приложениям эффективно использовать данные, распределенные между несколькими репозиториями
Интеграция процессов	Как управлять исполнением распределенного бизнеспроцесса, если отдельные бизнес-функции выполняются разными приложениями
Портал-ориентированная интеграция	Как конечные пользователи могут эффективно решать задачи, требующие доступа к информации, распределенной между несколькими приложениями

Агрегация сущностей

Агрегация сущностей — наиболее доступный, а поэтому и наиболее распространенный вид интеграции.

Цель агрегации сущностей — обеспечить приложениям возможность эффективно использовать данные, распределенные между несколькими репозиториями. Для эффективной работы с данными приложениям необходимо иметь единое согласованное в масштабах организации (отрасли) представление ключевых сущностей (например, таких как «Клиент», «Заказ», «Счет», «Продукт» и т.п.).

Агрегация сущностей

Задача осложняется тем, что:

- корпоративные системы, как правило, используют разные информационные модели для описания одной и той же сущности (например, сущность «сотрудник» по-разному представлена в бухгалтерской и кадровой системах);
- зачастую существует семантический диссонанс между данными из разных систем. Один и тот же элемент данных из разных систем может представлять разную информацию (например, атрибут NumberOfDaysRemaining для задачи проекта может содержать только рабочие дни в одном репозитории или включать выходные дни в другом);
- даже при отсутствии семантических различий возможно несовпадение данных для одного и того же экземпляра сущности (например, адрес одного и того же клиента в системе оформления заказа и адрес в CRM-системе различаются);
- репозитории могут содержать ошибочные данные;
- при проведении синхронизации данных между несколькими репозиториями может быть нарушена ссылочная целостность;
- приложение-потребитель может затребовать данные, хранящиеся в разных репозиториях.

Промежуточный слой

Решением является создание промежуточного слоя, обеспечивающего единое логическое представление сущностей в масштабе организации и физически связанного как с репозиториями данных, так и с приложениями — потребителями информации

Пример архитектуры ИС



Основные этапы агрегации сущностей

Агрегация сущностей вне зависимости от конкретной реализации интеграционного решения включает два основных этапа:

- определение единой в рамках предприятия модели данных, которая обеспечит унифицированное представление сущностей;
- установление физических связей между компонентами интеграционного решения.

Промежуточный слой должен

В разных репозиториях используются разные схемы данных для одной и той же сущности. Задача промежуточного слоя — гармонизировать различия между этими схемами. Промежуточный слой должен:

- удовлетворять потребностям всех интегрируемых приложений;
- определять каноническую схему данных для всех сущностей, используемых несколькими приложениями;
- поддерживать трансформации схем каждого источника данных в каноническую схему данных.

Архитектурные подходы

Существуют два базовых архитектурных подхода к интеграции информации:

- репликация данных;
- федерация информации.

Репликация данных

- Под репликацией данных понимают процесс перемещения данных между двумя или более репозиториями. Репликация — это ориентированная на обработку наборов данных технология преобразования, предназначенная для решения задач миграции, консолидации, создания хранилищ данных.

Репликация данных

Технология репликации ориентирована на обработку очень больших объемов данных.

- Основными достоинствами репликации являются:
- возможность использования разнородных данных любого качества (поскольку на этапе трансформации могут быть выполнены все необходимые процедуры проверки, очистки и обогащения данных);
- хорошая масштабируемость решений;
- наличие четкой методологии интеграции;
- асинхронный обмен данными;
- отсутствие необходимости изменений в интегрируемых информационных системах;
- эффективный доступ к интегрируемым данным за счет отделения приемника от источника;
- слабая связь между интегрируемыми системами;
- возможность повторного использования объектов и преобразований (снижает затраты на разработку и поддержку);
- относительно низкая стоимость;
- наличие доступных инструментов (программного обеспечения класса middleware).

Федерация информации

Под федерацией информации понимают процесс извлечения данных из различных источников в режиме реального времени и представление их в едином унифицированном виде. Это технология прозрачного доступа и преобразования данных, обеспечивающая единый интерфейс доступа ко всей корпоративной информации.

Достоинствами федерации информации являются:

- возможность интеграции структурированных и неструктурированных данных из многих источников;
- доступ к данным в режиме реального времени (по требованию);
- поддержка процессов чтения и записи данных;
- возможность преобразования данных для бизнес-анализа и обмена информацией;
- наличие доступных инструментов (программного обеспечения класса middleware).

Сравнительная характеристика ПОДХОДОВ К ИНТЕГРАЦИИ ДАННЫХ

Критерий сравнения	Репликация	Федерация
Поток данных	В одну сторону — от источника к приемнику	В обе стороны
Перемещение данных	По расписанию — пакетная загрузка; управляется процессом	В момент запроса; управляется запросом SQL
Задержка	День-неделя-месяц	Реальное время
Преобразование, очистка, использование метаданных в процессе	Лучшее; обычно высокая степень повторного использования объектов и процессов	Среднее; преобразования встроены в представления и объекты базы данных
Транспорт	FTP, прямое соединение с базой данных	Прямое соединение с базой данных
Объемы обрабатываемых данных	Очень большие (миллионы записей)	Средние (сотни тысяч удаленных записей)
Сложность преобразований	Любая	Преобразования, которые можно описать на SQL
Поддержка мониторинга событий	Ограниченная; можно улучшить с помощью технологии захвата изменений в источнике	Зависит от возможностей триггеров в источнике
Контроль потоков работ	По расписанию; обработка ошибок	Нет

Выбираем источник и приемник

Для каждого случая интеграции данных важно четко определить, какая система будет источником, а какая – приемником.

Например, у вас есть система CRM и программа 1С: Торговля. В обеих системах существует такое понятие, как контактное лицо. В принципе, вводить его вы можете и с одной, и с другой стороны. В данном случае, очевидно, что источником стоит назначить CRM, так как этого требует логика работы с любой CRM-системой.

Аналогично и в других случаях. Нужно понимать, в какой системе пользователь будет вводить данные, а какая станет получателем этих данных через интеграцию. Это обязательно согласовывается с клиентом (пользователем), кроме случаев, когда источник очевиден. При этом обязательно нужно поставить в известность клиента, что данные определенного типа следует вводить именно через систему-источник.

Сопоставление объектов (данных)

Каждый раз при работе с данными нужно очень хорошо понимать, что именно вы выгружаете, в каком виде, а также, куда вы будете выгружать эти данные. В некоторых случаях в источнике у вас будет строковая переменная, а в приемнике – два или более объектов. В других важно просто правильно выбрать объект-приемник.

Например, практически в любой CRM контактное лицо и клиент – это одно и то же. С другой стороны в 1С контактное лицо может быть клиентом, партнером, поставщиком. И очень важно понимать, куда именно записывать данные этого контактного лица. Также важно сопоставлять все данные до того, как начнется работа непосредственно с кодом. Для этого прекрасно подойдут таблицы или блок-схемы.

Также очень важно понимать, какие преобразования потребуются для выгружаемых данных. Например, нужные для интеграции данные в источнике хранятся в качестве перечисления в виде текста. А в приемнике (пусть это будет 1С) аналогичное перечисление имеет ссылочный тип. Следовательно, вам потребуется преобразовать текст в ссылку, и уже ссылку передать в документ.